

# Understanding System Call Overhead

<https://gemini.google.com/app/fb55d428bd286c5d>

User prompt: the overhead of system calls itself. meaning?

Response: **"The overhead of system calls itself"** refers to the fixed time and CPU performance cost incurred **just by making a system call**, regardless of how much actual work the requested operation takes to execute.

When a program asks the operating system (OS) to perform a privileged task—such as reading a file, sending a network packet, or allocating memory—the CPU must safely transition from standard user mode to privileged kernel mode. That transition process creates performance friction.

## Core Sources of Overhead

- **Mode Switching (Context Switch):** The CPU must save current user-space registers and program state, switch control registers, and jump to OS kernel code, then restore the user state upon completion.
- **CPU Cache & Pipeline Invalidation:** Jumping execution context flushes CPU instruction pipelines and often causes cache misses (since kernel code and data reside in different address spaces).
- **Security & Isolation Barriers:** Hardware mitigation mechanisms (e.g., against Spectre/Meltdown vulnerability exploits like KPTI) slow down mode transitions further by isolating memory tables.
- **Parameter Validation:** The kernel must sanitize, copy, and verify pointers passed from user space to prevent bad actors or buggy software from crashing the system.

## Comparison: Function Call vs. System Call

| Metric          | Standard Function Call (user-space) | System Call (kernel-space)                                     |
|-----------------|-------------------------------------|----------------------------------------------------------------|
| Execution Space | Same process memory space           | Crosses user → kernel boundary                                 |
| CPU Instruction | Simple CALL / RET                   | CPU interrupt or dedicated trap instruction (SYSCALL/SYSENTER) |
| Latency Cost    | A few nanoseconds                   | Hundreds to thousands of nanoseconds                           |

## Real-World Example

If an application needs to read 1 MB of data:

- **Bad Approach:** Making 1,000,000 individual `read()` system calls (1 byte per call). The CPU spends 99% of its time managing system call overhead rather than transferring data.
- **Optimized Approach:** Making 1 `read()` system call for a 1 MB chunk (or using user-space buffering like `fread()`). System call overhead is paid only once.

User prompt: tell me need of io\_uring and what uring mean and explain what problems it solves and how it works in detail?

Response: `io_uring` is a high-performance Linux asynchronous I/O framework introduced in Linux 5.1 (2019) by Jens Axboe. The name `uring` stands for **User Ring** (or **I/O Ring Buffer**)—referring to shared ring buffer data structures mapped between user space and kernel space.

## The Need for io\_uring & Problems Solved

Before `io_uring`, Linux lacked a unified, truly asynchronous I/O interface that performed well across both disk storage and network sockets.

- **Heavy System Call Overhead:** Traditional readiness notification models like `epoll` require two system calls per operation (e.g., `epoll_wait()` to learn a socket is ready, then `read()` to fetch data). At millions of operations per second, mode switches bottleneck the CPU.
- **Limitations of Linux Async I/O (aio):** Linux's older `io_submit()` mechanism only worked for direct I/O (`O_DIRECT` disk files), blocking unpredictably if buffered I/O or network sockets were passed.
- **Buffer Copying Costs:** Conventional I/O requires copying data back and forth between kernel buffers and user memory spaces.

`io_uring` solves these by enabling **zero-syscall (or batched) asynchronous operations** across *all* file descriptors (files, sockets, pipes, and devices).

## Architecture & How `io_uring` Works

`io_uring` replaces individual read/write system calls with lockless ring buffers allocated in memory shared between the application and the Linux kernel.

### 1. The Two Ring Buffers

- **Submission Queue (SQ):** The application writes I/O requests (called **SQEs** or Submission Queue Entries) to the tail of this circular ring. The kernel reads from the head to process them.
- **Completion Queue (CQ):** The kernel writes completion results (called **CQEs** or Completion Queue Entries) to the tail of this ring. The application reads from the head to consume completed results.

### 2. Lockless Ring Operation

Because head and tail pointers are modified by opposite sides (app updates SQ tail, kernel updates SQ head), producers and consumers operate concurrently without mutex lock contention. Memory barriers ensure cache synchronization across CPU cores.

### 3. Execution Modes

- **Batched Mode (Default):** The application fills multiple SQEs in user space without entering the kernel. It then calls `io_uring_enter()` **once** to submit the whole batch and optionally wait for completed CQEs.
- **Kernel Polling Mode (`IORING_SETUP_SQPOLL`):** A dedicated kernel thread continuously polls the SQ ring for new work. The application simply places requests in the ring without executing any system calls at all—achieving true **zero-syscall** I/O.
- **IO Polling Mode (`IORING_SETUP_IOPOLL`):** For ultra-fast NVMe storage, the kernel polls the hardware block driver directly instead of waiting for hardware interrupts, reducing latency to physical device limits.

*User prompt: explain all the needs of `io_uring` in detail in simple language*

Response: To understand why `io_uring` was created, imagine running a busy restaurant.

In a traditional setup, every time a waiter wants to send an order to the kitchen, they have to physically walk through a heavy security door, pass the slip to the chef, wait for confirmation, and walk back. Walking back and forth through that door takes up most of the waiter's energy.

`io_uring` replaces that system with a **shared clipboard** mounted on the wall. The waiter just writes orders on the clipboard, and the chef reads them whenever they are ready—nobody has to walk through the security door anymore.

Here are all the major reasons (**needs**) why Linux required `io_uring`:

### 1. Eliminating System Call Overhead ("The Heavy Security Door")

- **The Problem:** Every time your program calls `read()`, `write()`, or `send()`, the CPU must execute a **system call**. This pauses your application and switches the CPU from "User Mode" to "Kernel Mode." As network speeds and SSDs became hyper-fast (millions of requests per second), CPUs spent more time switching back and forth than actually processing data.

- **The Need:** A way to ask the operating system to do work **without having to trigger a system call for every single operation**.

2. Fixing the Limitations of Older Linux Tools

Before `io_uring`, developers relied on two main tools, but both had major drawbacks:

- **epoll (Readiness Model):** `epoll` doesn't do the I/O for you; it only notifies you when a socket is ready. This means you need **two steps**:
  1. Ask `epoll`: *"Is socket #5 ready to read?"* (System Call 1)
  2. Perform `read()`: *"Okay, read the data from socket #5."* (System Call 2)At high scale, double the system calls meant half the efficiency.
- **Linux aio (Asynchronous I/O):** Linux had an older async system called `aio`, but it was notoriously limited. It **only worked for unbuffered disk reads/writes** (`O_DIRECT`). If you tried to use it on regular files, network sockets, or pipes, it would silently block your application anyway.
- **The Need:** A single, unified framework that handles **everything** asynchronously—network sockets, regular files, pipes, and devices.

3. Batching Multiple Requests at Once

- **The Problem:** If a Web server receives 100 HTTP requests at the same time, traditional systems submit them one by one.
- **The Need:** A mechanism to pack 100 requests into a queue in memory all at once, and tell the kernel: *"Here are 100 tasks, handle them in the background and let me know when they're done."*

4. Supporting High-Speed Modern Hardware (NVMe SSDs & 100Gb Networks)

- **The Problem:** Modern NVMe drives can perform millions of Input/Output Operations Per Second (IOPS). Hardware interrupts (where the disk alerts the CPU that a read is finished) actually slow down the CPU at these speeds because interrupts fire constantly.
- **The Need:** Polling support (`IORING_SETUP_IOPOLL`). Instead of waiting for hardware interrupts, `io_uring` allows the OS kernel to continuously poll high-speed hardware directly, reducing latency to raw hardware limits.

5. Achieving True "Zero System Call" Execution

- **The Problem:** Even with batching, you still need to call the kernel occasionally to process the queue.
- **The Need:** `io_uring` introduced a special kernel thread mode (`SQPOLL`). In this mode, a background kernel thread continually monitors your shared memory queue. Your application can queue thousands of read/write requests, and the kernel processes them **without your application ever executing a single system call**.

Summary Table

| Operational Need | Old Linux Approach                                                             | io_uring Approach                                            |
|------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------|
| Network Sockets  | <code>epoll</code> + <code>read()/write()</code> (Multiple system calls)       | Unified async submission ring                                |
| Disk Files       | Linux <code>aio</code> (Only worked on <code>O_DIRECT</code> unbuffered files) | Fully async for all file types & operations                  |
| System Call Cost | Paid on every single read/write                                                | Batched, or completely <b>0</b> system calls in polling mode |
| Efficiency       | CPU bottlenecked by mode switching                                             | CPU focused entirely on application logic                    |

User prompt: what is kernel and user space?

Response: **Kernel space** and **user space** are the two fundamental memory divisions created by an operating system to isolate application software from critical system hardware.

This division relies on hardware protection levels provided by the CPU (such as x86 privilege rings: Ring 0 for kernel space and Ring 3 for user space) to keep the system secure and stable.

Core Characteristics

| Attribute         | User Space                                                     | Kernel Space                                                     |
|-------------------|----------------------------------------------------------------|------------------------------------------------------------------|
| Privilege Level   | Low privilege (unprivileged mode / Ring 3)                     | Highest privilege (supervisor mode / Ring 0)                     |
| What Runs Here    | Applications (Web browsers, games, databases, Python scripts)  | Operating system core, device drivers, system services           |
| Hardware Access   | Strictly prohibited from direct hardware access                | Unrestricted access to CPU, RAM, disk, and network interfaces    |
| Memory Isolation  | Processes are sandboxed; cannot read/write each other's memory | Full visibility and control over all virtual and physical memory |
| Fault Consequence | App crashes (e.g., Segmentation Fault), system stays running   | Entire system crashes or halts (e.g., Kernel Panic, BSOD)        |

Why the Separation Exists

- 1. **System Stability:** If a user-space application encounters a bug or memory leak, only that specific process crashes. The operating system remains operational.
- 2. **Security & Sandboxing:** Prevents malicious programs from touching protected physical memory, sniffing network traffic from other apps, or overwriting critical OS files.
- 3. **Resource Arbitration:** Ensures multiple applications safely share limited CPU, memory, and disk resources through OS management.

How They Interact

Because user-space applications cannot touch hardware directly, they must request the kernel to perform operations on their behalf:

- 1. **System Call Request:** A program in user space invokes a **system call** (e.g., `read()`, `write()`, `open()`).
- 2. **Mode Switch:** The CPU pauses the application, transitions from Ring 3 to Ring 0, and switches to the kernel context.
- 3. **Execution:** The kernel validates the request parameters and safely interfaces with the underlying hardware drive or network card.
- 4. **Return:** The CPU switches back from Ring 0 to Ring 3, returning control and data to the user-space application.

User prompt: this repo is uodated on github how to pull it - <https://github.com/Pallav-Biyala/CS331-T008-NetworkIO>

Response: Open Git Bash or your terminal inside that local directory and run the following commands:

1. Navigate & Pull Updates

```
git pull origin main

(If the default branch uses master instead, run git pull origin master)
```

2. Handling Uncommitted Local Changes (If Git throws an error)

If you edited files locally and Git prevents the pull, resolve it using one of these options:

- **Option A: Keep local edits and merge updates**

```
git stash
git pull origin main
git stash pop
```

- **Option B: Discard local edits and match GitHub exactly**

```
git fetch origin
git reset --hard origin/main
```

## How to Open Terminal Directly in File Explorer

1. Click the address bar at the top of your File Explorer window (where it shows CS331-T008-NetworkIO).
  2. Type `cmd` or `bash` and press **Enter**.
  3. Run `git pull origin main`.
- 

User prompt: *what is a header file .h?*

Response: A **header file** (ending with `.h` or `.hpp` in C and C++) is a source file containing declarations of functions, macros, structures, and classes without their actual execution logic.

It acts as an interface or contract between different parts of a program, allowing multiple code files to share functions and data types safely.

## Core Purpose of a Header File

1. **Forward Declarations:** Tells the compiler *what* a function or variable looks like (its name, return type, and parameters) before it is actually used.
2. **Code Reusability & Organization:** Allows you to define interfaces once and share them across multiple `.c` or `.cpp` implementation files.
3. **Faster Compilation:** Changes to implementation logic inside a `.c` file do not require re-compiling every file that includes the `.h` file—only the modified file needs to be recompiled and re-linked.

## Structure of a Header File

A typical header file uses **include guards** to prevent header content from being defined multiple times during compilation.

### math\_utils.h (Header File)

```
#ifndef MATH_UTILS_H    // Include Guard (prevents double inclusion)
#define MATH_UTILS_H

// Struct Definition
typedef struct {
    int x;
    int y;
} Point;

// Function Declarations (Prototypes)
int add(int a, int b);
int multiply(int a, int b);

#endif
```

### math\_utils.c (Implementation File)

```
#include "math_utils.h"

// Actual Function Implementations
int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

## Difference Between Angle Brackets <> and Quotes ""

When including header files in your code using `#include`:

- `#include <stdio.h>` (**Angle Brackets**): Instructs the preprocessor to search system standard library paths (e.g., standard C library files).
  - `#include "math_utils.h"` (**Quotes**): Instructs the preprocessor to search the local directory of your current project first before checking system paths.
- 

*User prompt: how do we create a new branch?*

Response: You can create a new branch directly on GitHub or locally from your terminal using Git commands.

### Method 1: On GitHub (Directly from your screen)

1. Click on the input box that currently says `Find or create a branch...` inside the open dropdown menu.
2. Type your new branch name (e.g., `epoll-server` or `feature-dev`).
3. Click the blue button below that says `Create branch: <your-branch-name> from 'select-server'`.

### Method 2: From Terminal / Git Bash (Recommended)

Open your terminal inside the project folder and run:

#### 1. Create and switch to the new branch in one step:

```
git checkout -b <your-new-branch-name>
```

(Or in newer Git versions: `git switch -c <your-new-branch-name>`)

#### 2. Push the new branch to GitHub:

```
git push -u origin <your-new-branch-name>
```

---

*User prompt: I have creetd a new branch but not added in github*

Response: To publish your new `io_uring` branch to GitHub, run this command in your PowerShell terminal:

```
git push -u origin io_uring
```

### What happens next:

1. `-u origin io_uring` creates the `io_uring` branch on GitHub and links your local branch to it.
  2. Refresh your browser page on GitHub, and `io_uring` will appear in the branch dropdown list.
  3. For future updates on this branch, you can just run `git push`.
-

User prompt: can we know who created which branch in github?

Response: Yes, you can track down who created a branch using a few different methods on GitHub and via the command line:

### Method 1: Check the GitHub Web UI (Easiest)

1. Click on **View all branches** (or click **3 Branches** next to the branch dropdown).
2. Look at the branch list:
  - **In the "Your branches" tab:** GitHub automatically groups branches created by your account.
  - **In the "All branches" tab:** Beside each branch name, GitHub shows the profile avatar and name of the author who pushed the **latest commit** or opened a pull request on that branch.

### Method 2: Check via Terminal / Git CLI

To view the author associated with the latest commit on remote branches, run:

```
git for-each-ref --format='% (commitdate:short) %09 % (authorname) %09 % (refname:short)' refs/remotes/
```

*This prints the last commit date, the author's name, and the branch name for all remote branches.*

### Method 3: GitHub Activity Feed / Audit Logs

If you need to know the **exact person who created the branch reference** (even if someone else pushed a commit later):

- Go to **Activity** or **Insights > Network** on the GitHub repository page to view the graphical history of where and when branches diverged.
- If you are an administrator of an **Organization**, check **Settings > Audit log** to search specifically for `repo.create_ref` events to see the exact user action.

User prompt: in what all parameters io\_uring is better than epoll

Response: io\_uring outperforms epoll across several performance, architecture, and design parameters:

### 1. System Call Count & Overhead (The Biggest Advantage)

- **epoll:** Requires a **minimum of 2 system calls** per I/O cycle. First, `epoll_wait()` blocks or returns ready events, then individual calls like `read()`, `write()`, `accept()`, or `send()` perform the actual data transfer.
- **io\_uring:** Can achieve **0 system calls per I/O** in polling mode (`IORING_SETUP_SQPOLL`). In standard batching mode, it submits dozens or hundreds of I/O operations and retrieves completions using a **single `io_uring_enter()` call**.

### 2. Execution Paradigm: Readiness vs. Completion

- **epoll (Readiness-based):** Notifies you when a file descriptor is **ready to be read/written**. Your application must still initiate and execute the transfer operation in user space.
- **io\_uring (Completion-based):** Takes your request, executes the read or write in the background inside the kernel, and notifies you **when the I/O is fully complete**.

### 3. File & Device Support

- **epoll:** Only works on **non-blocking, pollable file descriptors** (network sockets, pipes, eventfds, timers). It **fails on regular disk files** (regular file operations on Linux always block regardless of non-blocking flags).
- **io\_uring:** Provides a **unified asynchronous interface for everything**—network sockets, regular files, block devices, pipes, and inter-process communication.

### 4. Memory Copying & Zero-Copy Efficiency

- **epoll**: Data buffer references must be passed across system call boundaries for every operation, triggering kernel-to-user buffer validation and copying.
- **io\_uring**: Uses **fixed pre-registered buffers** (`IORING_REGISTER_BUFFERS`) and pre-registered file descriptors. The kernel maps these buffers once, enabling true **zero-copy network and storage I/O** with minimal lookup overhead.

5. Latency and CPU Cache Locality

- **epoll**: High rate of user-to-kernel mode switches flushes CPU pipelines, increases instruction cache misses, and incurs hardware CPU security mitigation penalties (e.g., KPTI for Meltdown/Spectre).
- **io\_uring**: Reduces context switches, keeping CPU caches warm and maximizing CPU pipeline efficiency under high throughput.

6. Hardware Driver Polling (Kernel & NVMe Integration)

- **epoll**: Relies entirely on traditional hardware interrupts to awaken the event loop when network or disk data arrives.
- **io\_uring**: Supports **IO Polling** (`IORING_SETUP_IOPOLL`), where the kernel polls fast storage drivers (like NVMe) directly. This bypasses interrupt handling latency altogether for ultra-low latency setups.

Summary Comparison Table

| Parameter               | epoll                            | io_uring                  |
|-------------------------|----------------------------------|---------------------------|
| Model                   | Readiness-based                  | Completion-based          |
| Syscalls per I/O        | $\geq 2$                         | 1 (Batched) or 0 (SQPOLL) |
| Regular Files Support   | ✗ No (Blocks)                    | ✓ Yes (Fully Async)       |
| Network Sockets Support | ✓ Yes                            | ✓ Yes                     |
| Zero-Copy Features      | Limited                          | Native registered buffers |
| Polling Modes           | ✗ No                             | ✓ SQPOLL & IOPOLL         |
| CPU Cache Overhead      | High (frequent context switches) | Low (shared memory rings) |

User prompt: compare all these parametes?

Response: These table headers correspond to standard network I/O benchmarking metrics used to evaluate and compare event engines (select, poll, epoll, and io\_uring).

Table 1: Throughput & Capacity Metrics

| Parameter      | Meaning                                                                         | Higher/Lower is Better | Why It Matters                                                                                             |
|----------------|---------------------------------------------------------------------------------|------------------------|------------------------------------------------------------------------------------------------------------|
| Engine         | The I/O multiplexing mechanism evaluated (e.g., select, poll, epoll, io_uring). | N/A                    | Identifies which system implementation is being tested.                                                    |
| Conns          | Active Concurrent Connections (concurrent open client sockets).                 | Higher is better       | Measures how well an engine scales under high concurrency (e.g., 1, 000 to 10, 000+ simultaneous clients). |
| BW Down (Mbps) | Download Bandwidth / Throughput in Megabits per second.                         | Higher is better       | Rates data delivery speed from server to client.                                                           |

| Parameter                   | Meaning                                                          | Higher/Lower is Better | Why It Matters                                                         |
|-----------------------------|------------------------------------------------------------------|------------------------|------------------------------------------------------------------------|
| <b>BW Up (Mbps)</b>         | Upload Bandwidth / Throughput in Megabits per second.            | Higher is better       | Rates data reception speed from client to server.                      |
| <b>Pkt Rate Down (kpps)</b> | Packet Rate Downward in <b>thousands of packets per second</b> . | Higher is better       | Measures how efficiently the stack pushes network packets downstream.  |
| <b>Pkt Rate Up (kpps)</b>   | Packet Rate Upward in <b>thousands of packets per second</b> .   | Higher is better       | Measures how efficiently the stack processes incoming network packets. |

**Table 2: System Call Performance Metrics**

| Parameter             | Meaning                                                                                                                                          | Higher/Lower is Better | Why It Matters                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------------------------------------------------------------------------|
| <b>Engine</b>         | The event driver being measured.                                                                                                                 | N/A                    | Baseline identifier for the target system architecture.                       |
| <b>Conns</b>          | Concurrent Connections test load.                                                                                                                | N/A                    | Establishes the scale of the workload during measurement.                     |
| <b>Syscall</b>        | The specific Linux system call executed (e.g., <code>epoll_wait</code> , <code>read</code> , <code>write</code> , <code>io_uring_enter</code> ). | N/A                    | Pinpoints which operation causes performance bottlenecks.                     |
| <b>Calls</b>          | Total number of system call invocations during the benchmark run.                                                                                | <b>Lower is better</b> | Fewer calls mean reduced context switching between user mode and kernel mode. |
| <b>Time (us/call)</b> | Average execution time per system call in <b>microseconds</b> ( $\mu s$ ).                                                                       | <b>Lower is better</b> | Measures kernel-side efficiency and execution latency for each call.          |

**Table 3: Resource Efficiency Metrics**

| Parameter               | Meaning                                                                    | Higher/Lower is Better | Why It Matters                                                                          |
|-------------------------|----------------------------------------------------------------------------|------------------------|-----------------------------------------------------------------------------------------|
| <b>Engine</b>           | The event driver being measured.                                           | N/A                    | Baseline identifier.                                                                    |
| <b>Conns</b>            | Concurrent Connections test load.                                          | N/A                    | Test load indicator.                                                                    |
| <b>Context Switches</b> | Total CPU switches between user mode and kernel mode (or thread switches). | <b>Lower is better</b> | Context switches flush CPU instruction pipelines and cache lines, reducing performance. |
| <b>RSS Diff (KB)</b>    | Resident Set Size difference in <b>Kilobytes</b> .                         | <b>Lower is better</b> | Measures the physical RAM overhead added by the engine to manage the connections.       |

**Summary of Engine Expectations**

- `select` / `poll`: High **Calls**, high **Context Switches**, linear  $O(N)$  CPU scaling as **Conns** increases.
- `epoll`: Low CPU scaling  $O(1)$ , moderate **Calls** and **Context Switches** (minimum 2 calls per I/O event).
- `io_uring`: Lowest **Calls** (batched or 0 via `SQPOL`), minimal **Context Switches**, highest **BW** and **kpps** throughput under high connection loads.

User prompt: comaprw all these parameters for epoll and io uring

Response: 1. Throughput & Packet Processing (BW Down/Up, Pkt Rate Down/Up)

- **epoll:** Offers high throughput for standard network applications. However, at ultra-high packet rates (hundreds of thousands of packets per second), performance plateaus because the CPU spends significant processing time in mode-switching overhead (s) and context switches.
- **io\_uring:** Achieves higher overall throughput (**BW**) and higher packet processing rates (**kpps**) under heavy client concurrency. By batching operations into submission entries and leveraging zero-copy registered buffers, it maximizes line-rate packet processing on high-speed network interfaces.

2. System Call Count (Calls)

- **epoll:** Requires multiple system calls per event cycle. A typical transaction requires `epoll_wait()` to discover readiness, followed by `recv()` or `send()` calls to process the data—yielding a high system call count that scales linearly with activity.
- **io\_uring:** Dramatically reduces system calls. Applications batch dozens or hundreds of requests in the submission queue ring buffer (SQ) and execute them via a single `io_uring_enter()` call. In kernel polling mode (SQPOLL), the system call count drops to **0** during active workload streaming.

3. Execution Latency (Time us/call)

- **epoll:** Individual system calls take relatively little time to process in isolation (low microseconds per call). However, because an application must issue separate calls to check readiness and perform reads or writes, total end-to-end request latency accumulates.
- **io\_uring:** Individual `io_uring_enter()` calls take slightly longer in execution time ( $\mu s$ ) because the kernel processes an entire array of submitted I/O operations in one pass. However, because the cost is amortized across many queued requests, the **average latency per individual I/O operation is significantly lower**.

4. CPU Context Switches (Context Switches)

- **epoll:** Frequent transitions between user space and kernel space trigger high context-switching overhead. This causes continuous CPU cache invalidations, instruction pipeline flushes, and higher latency under heavy load.
- **io\_uring:** Shared memory ring buffers allow user applications and the kernel to communicate locklessly without constant privilege boundary transitions. This keeps CPU instruction and data caches warm, resulting in far fewer context switches.

5. Memory Consumption (RSS Diff KB)

- **epoll:** Uses minimal memory per connection. The kernel tracks registered descriptors using an internal red-black tree (`struct epitem`), requiring modest physical memory (RSS) footprints even with tens of thousands of idle connections.
- **io\_uring:** Has a slightly higher initial memory footprint (RSS) because it pre-allocates contiguous shared ring buffers (SQ and CQ entries) and memory-mapped ring pages upon initialization. While it consumes marginally more fixed physical RAM per ring instance, it eliminates repetitive user-to-kernel buffer allocation overhead during runtime.

Direct Parameter Comparison

| Metric / Parameter      | epoll Behavior                                     | io_uring Behavior                                     | Winner   |
|-------------------------|----------------------------------------------------|-------------------------------------------------------|----------|
| BW Down/Up (Mbps)       | Moderate to high; degrades at saturation           | Maximized throughput under heavy loads                | io_uring |
| Pkt Rate Down/Up (kpps) | Lower due to per-packet system call boundaries     | Significantly higher due to batched requests          | io_uring |
| Calls (Syscall Count)   | High (at least 2–3 calls per I/O event)            | Extremely low (batched) or 0 (SQPOLL)                 | io_uring |
| Time (us/call)          | Fast per individual call, but higher total latency | Higher time per enter call, but lower latency per I/O | io_uring |

| Metric / Parameter | epoll Behavior                           | io_uring Behavior                                  | Winner   |
|--------------------|------------------------------------------|----------------------------------------------------|----------|
| Context Switches   | High (frequent Ring 3 ↔ Ring 0 switches) | Minimal (shared lockless memory rings)             | io_uring |
| RSS Diff (KB)      | Lower base RAM footprint per connection  | Slightly higher pre-allocated ring memory overhead | epoll    |